

**LAPORAN PRAKTIKUM 2**  
**STRUKTUR DATA ARRAYLIST**

**Disusun Oleh:**

Dinda Amelia

(2511531020)

**Dosen Pengampu:**

Dr. Wahyudi, S.T, M.T

**Asisten Praktikum:**

Muhammad Galid Avero



**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**  
**PADANG 2026**

## **KATA PENGANTAR**

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang ARRAYLIST Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memberikan pemahaman mengenai cara membuat program lebih rapi, mudah dipahami, fleksibel, dan aman.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

## DAFTAR ISI

|                                                   |    |
|---------------------------------------------------|----|
| KATA PENGANTAR.....                               | i  |
| DAFTAR ISI .....                                  | ii |
| BAB I.....                                        | 1  |
| PENDAHULUAN .....                                 | 1  |
| 1.1 Latar Belakang .....                          | 1  |
| 1.2 Tujuan .....                                  | 1  |
| 1.3 Manfaat Praktikum .....                       | 2  |
| BAB II.....                                       | 3  |
| PEMBAHASAN.....                                   | 3  |
| 2.1 Dasar Teori.....                              | 3  |
| 2.2 Langkah kerja.....                            | 4  |
| 2.2.1 Buka eclipse.....                           | 4  |
| 2.2.2 Membuat Java Project .....                  | 4  |
| 2.2.3 Class 1 “ArrayList1_2511531020” .....       | 5  |
| 2.2.4 Class 2 “Mahasiswa_2511531020” .....        | 8  |
| 2.2.5 Class 3 “MahasiswaDriver_2511531020”.....   | 10 |
| 2.2.6 Class 4 “DaftarKata_2511531020” .....       | 14 |
| 2.2.7 Class 5 “DaftarKataDriver_2511531020” ..... | 19 |
| 2.3 Tugas Praktikum.....                          | 23 |
| 2.3.1 Class Musik_2511531020 .....                | 23 |
| 2.3.2 Class Playlist_2511531020.....              | 26 |
| BAB III.....                                      | 32 |
| KESIMPULAN .....                                  | 32 |
| DAFTAR PUSTAKA.....                               | 33 |

## **BAB I**

### **PENDAHULUAN**

#### **1.1 Latar Belakang**

ArrayList merupakan salah satu kelas dalam *Java Collections Framework* yang dirancang untuk mengatasi keterbatasan array biasa yang bersifat statis. Array hanya memiliki ukuran tetap sejak pertama kali dibuat, sehingga sulit digunakan ketika jumlah data yang harus disimpan berubah-ubah. Untuk menjawab kebutuhan tersebut, ArrayList hadir sebagai struktur data berbasis array dinamis yang mampu menyesuaikan ukuran secara otomatis sesuai jumlah elemen yang ditambahkan atau dihapus. Dengan berada dalam paket `java.util` dan mengimplementasikan antarmuka `List`, ArrayList menyediakan berbagai metode siap pakai seperti `add()`, `remove()`, `get()`, dan `set()` yang memudahkan pengelolaan data. Selain itu, ArrayList mempertahankan urutan elemen sesuai penambahan, mendukung duplikasi, dan memberikan fleksibilitas tinggi dalam manipulasi data. Latar belakang inilah yang menjadikan ArrayList sangat penting dalam pengembangan aplikasi Java, terutama ketika diperlukan pengelolaan data yang dinamis dan efisien.

#### **1.2 Tujuan**

1. Melatih pemahaman konsep Object-Oriented Programming (OOP) seperti class, constructor, method, dan encapsulation.
2. Menguasai penggunaan ArrayList sebagai struktur data dinamis untuk menyimpan dan mengelola objek maupun string.
3. Membiasakan diri dengan operasi dasar pada ArrayList: menambah, menghapus, mengubah, menyisipkan, dan menampilkan elemen.
4. Membuat program interaktif berbasis menu menggunakan Scanner, do-while, dan switch-case untuk mengelola data mahasiswa.
5. Mengimplementasikan method overriding (`toString()`) agar objek dapat ditampilkan dengan representasi yang lebih informatif.

### **1.3 Manfaat Praktikum**

1. Memberikan keterampilan praktis dalam mengelola data secara dinamis menggunakan ArrayList, yang lebih fleksibel dibanding array biasa.
2. Membantu memahami bagaimana objek-objek Java dapat digunakan untuk merepresentasikan entitas nyata (misalnya mahasiswa).
3. Melatih kemampuan problem solving dengan membuat menu interaktif untuk CRUD (Create, Read, Update, Delete) data.
4. Menunjukkan pentingnya encapsulation dan modularisasi dalam membangun program agar lebih terstruktur dan mudah dipelihara.
5. Menjadi dasar untuk mengembangkan aplikasi yang lebih kompleks, misalnya sistem manajemen data, dengan konsep OOP dan struktur data yang kuat.

## BAB II

### PEMBAHASAN

#### 2.1 Dasar Teori

ArrayList merupakan salah satu kelas dalam Java Collections Framework yang dirancang untuk mengatasi keterbatasan array konvensional yang bersifat statis. Array hanya memiliki ukuran tetap sejak pertama kali dibuat, sehingga kurang fleksibel ketika jumlah data yang harus disimpan berubah-ubah. Untuk menjawab kebutuhan tersebut, ArrayList hadir sebagai struktur data berbasis array dinamis yang mampu menyesuaikan ukuran secara otomatis sesuai jumlah elemen yang ditambahkan atau dihapus. Dengan berada dalam paket `java.util` dan mengimplementasikan antarmuka `List`, ArrayList menyediakan berbagai metode siap pakai seperti `add()`, `remove()`, `get()`, `set()`, dan `size()` yang memudahkan pengelolaan data. ArrayList mempertahankan urutan elemen sesuai penambahan, mendukung duplikasi, dan memberikan fleksibilitas tinggi dalam manipulasi data. Keunggulan utama ArrayList adalah kemampuannya mengelola data secara dinamis, sementara kelemahannya terletak pada operasi penghapusan atau penyisipan di tengah list yang relatif lebih lambat karena elemen-elemen setelah indeks tersebut harus digeser. Dalam konteks pemrograman berorientasi objek, penggunaan ArrayList juga mencerminkan prinsip *encapsulation* karena data disimpan dalam objek dan diakses melalui method publik, serta *abstraction* karena programmer tidak perlu mengetahui detail implementasi internal array dinamis. Dengan karakteristik tersebut, ArrayList menjadi salah satu fondasi penting dalam pemrograman Java untuk membangun aplikasi yang membutuhkan pengelolaan data yang efisien, fleksibel, dan mudah dipelihara.

## 2.2 Langkah kerja

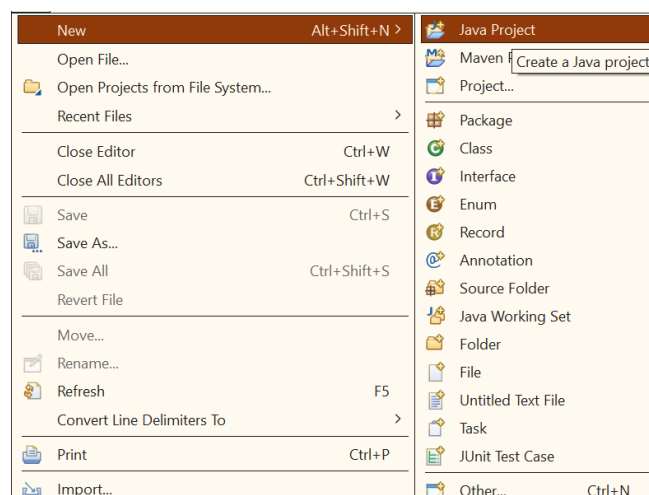
### 2.2.1 Buka eclipse



**Gambar 2.1** Buka eclipse jalankan

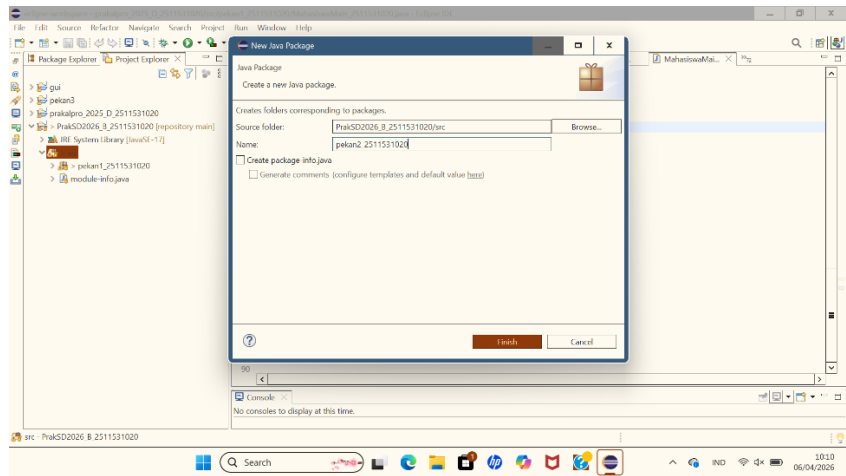
### 2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



**Gambar 2.2** Java Project

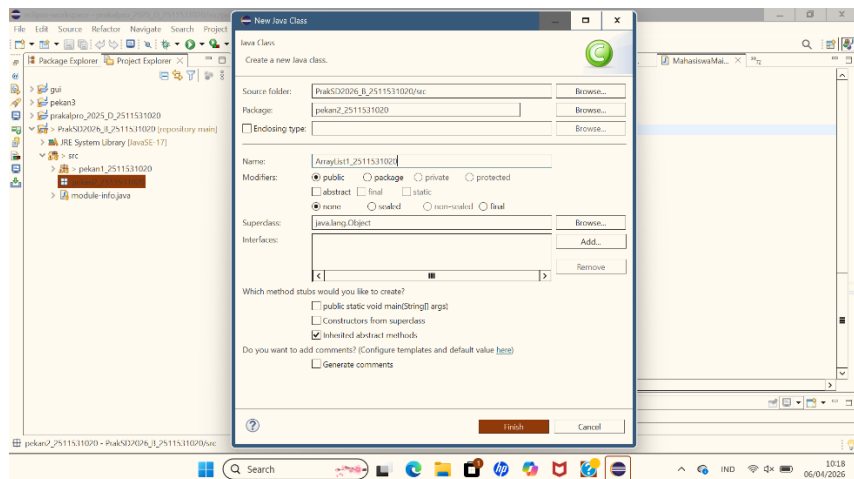
### 3. Buat nama project



**Gambar 2.3** Nama Project

### 2.2.3 Class 1 “ArrayList1\_2511531020”

1. Membuat package pekan2\_2511531020 dan class ArrayList1\_2511531020



**Gambar 2.4** Membuat Nama package dan class

2. Klik Finish
3. Kode Program



```

1 package pekan2_2511531020;
2
3 import java.util.ArrayList;
4
5 public class ArrayList1_2511531020 {
6
7     public static void main(String[] args) {
8         // Size of the ArrayList
9         int n = 5;
10
11         // Declaring the ArrayList with initial size n
12         ArrayList<Integer> arrli = new ArrayList<>(n);
13
14         //Appending new elements at the end of the list
15         for (int i = 1; i <= n; i++) {
16             arrli.add(i);
17         }
18
19         // printing elements
20         System.out.println(arrli);
21
22         // Remove elements at index 3
23         arrli.remove(3);
24
25         // Display the ArrayList
26         // after deletion
27         System.out.println(arrli);
28
29         // printing elements one by one
30         for (int i = 0; i < arrli.size(); i++) {
31             System.out.print(arrli.get(i) + " ");
32         }
33     }
34 }
35

```

**Gambar 2.5** Class 1 “ArrayList1\_2511531020

#### 4. Analisis Langkah

##### a. Tujuan Program

- Memahami penggunaan kelas ArrayList di Java.
- Melakukan operasi penambahan elemen, penghapusan elemen, serta menampilkan isi list.
- Menunjukkan fleksibilitas ArrayList dibandingkan array biasa.

##### b. Struktur Program

- Package: pekan2\_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: ArrayList1\_2511531020 → berfungsi sebagai kelas utama untuk menjalankan program.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.

##### c. Proses Eksekusi

- Deklarasi variabel n = 5 sebagai jumlah elemen yang akan ditambahkan.

- Membuat objek `ArrayList<Integer> arrli = new ArrayList<>(n)` dengan kapasitas awal lima.
- Menggunakan loop `for (int i = 1; i <= n; i++) arrli.add(i);` untuk menambahkan angka 1 sampai 5 ke dalam list.
- Setelah loop, isi list adalah [1, 2, 3, 4, 5].

d. Menampilkan Nilai List

- `System.out.println(arrli);` → mencetak seluruh isi list sekaligus.
- Output: [1, 2, 3, 4, 5].

e. Operasi Penghapusan Elemen

- `arrli.remove(3);` → menghapus elemen pada indeks ke-3 (yaitu angka 4).
- Isi list setelah penghapusan: [1, 2, 3, 5].

f. Menampilkan List Setelah Penghapusan

- `System.out.println(arrli);` → mencetak isi list setelah elemen dihapus.
- Output: [1, 2, 3, 5].

g. Menampilkan Elemen Satu per Satu

- Menggunakan loop `for (int i = 0; i < arrli.size(); i++) System.out.print(arrli.get(i) + " ");`.
- Output: 1 2 3 5.

h. Konsep OOP dan Struktur Data

- Encapsulation: `ArrayList` dibuat sebagai objek dengan konstruktor, menyembunyikan detail implementasi internal.
- Dynamic Data Structure: `ArrayList` dapat menambah dan menghapus elemen tanpa ukuran tetap.
- Method Utility: `add()`, `remove()`, `get()`, dan `size()` menunjukkan fungsi-fungsi bawaan untuk manipulasi data.
- Flexibilitas Output: Data dapat ditampilkan sekaligus atau satu per satu sesuai kebutuhan.

## 5. Output

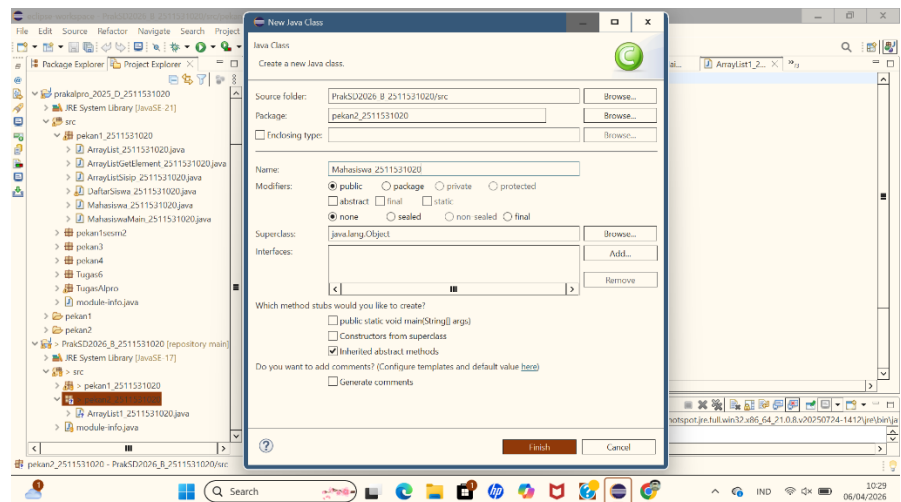


```
<terminated> ArrayList1_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

**Gambar 2.6 Output**

### 2.2.4 Class 2 “Mahasiswa\_2511531020”

1. Membuat package pekan2\_2511531020 dan class Mahasiswa\_2511531020



**Gambar 2.7 Membuat Nama package dan class**

2. Klik Finish
3. Kode Program

```

1 package pekan2_2511531020;
2
3 public class Mahasiswa_2511531020 {
4     String nim;
5     String nama;
6     String prodi;
7     Mahasiswa_2511531020(String nim, String nama, String prodi) {
8         this.nim = nim;
9         this.nama = nama;
10        this.prodi = prodi;
11    }
12
13    @Override
14    public String toString() {
15        return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;
16    }
17 }
18
19
20
21

```

**Gambar 2.8** Class 2 “Mahasiswa\_2511531020”

#### 4. Analisis Langkah

##### a. Tujuan Program

- Membuat kelas Mahasiswa sebagai representasi objek mahasiswa.
- Menyimpan data mahasiswa berupa NIM, nama, dan program studi.
- Menampilkan informasi mahasiswa dengan format yang rapi melalui method toString().

##### b. Struktur Program

- Package: pekan2\_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: Mahasiswa\_2511531020 → berfungsi sebagai model data mahasiswa.
- Variabel: nim, nama, prodi → atribut yang merepresentasikan identitas mahasiswa.
- Constructor: menerima parameter nim, nama, dan prodi untuk menginisialisasi objek.
- Method: toString() → menampilkan data mahasiswa dalam format string.

##### c. Proses Eksekusi

- Saat objek mahasiswa dibuat dengan `new Mahasiswa_2511531020("2511531020", "Dinda", "Informatika")`, constructor akan mengisi nilai atribut sesuai parameter.
- Perlu diperhatikan bahwa dalam kode terdapat kesalahan kecil: baris `this.nama = nim;` seharusnya `this.nim = nim;`. Jika tidak diperbaiki, nilai NIM tidak akan tersimpan dengan benar.
- Setelah objek terbentuk, pemanggilan `System.out.println(objekMahasiswa);` akan otomatis memanggil method `toString()` untuk menampilkan data mahasiswa.

#### d. Menampilkan Nilai Objek

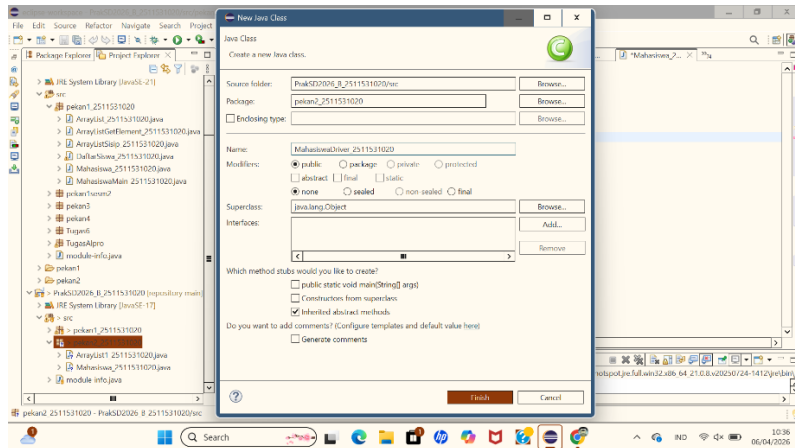
- Output yang dihasilkan: NIM: 2511531020, Nama: Dinda, Prodi: Informatika
- Format ini berasal dari implementasi `toString()` yang menggabungkan nilai atribut menjadi satu string.

#### e. Analisis Konsep OOP

- Encapsulation: atribut mahasiswa disimpan dalam objek, sehingga detail data terbungkus rapi.
- Constructor: digunakan untuk menginisialisasi nilai atribut saat objek dibuat.
- Method Overriding: `toString()` dioverride untuk memberikan representasi string yang lebih informatif.
- Object Representation: pemanggilan `System.out.println(objek)` memanfaatkan polymorphism untuk menampilkan isi objek dengan cara yang lebih bermakna.

### 2.2.5 Class 3 “MahasiswaDriver\_2511531020”

1. Membuat package `pekan2_2511531020` dan class `MahasiswaDriver_2511531020`



Gambar 2.9 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan2_2511531020;
2 import java.util.ArrayList;
3
4
5 public class MahasiswaDriver_2511531020 {
6     //1. Method untuk menampilkan menu
7     public static void tampilkanMenu_2511531020() {
8         System.out.println("\nMenu:");
9         System.out.println("1. Tambah Mahasiswa");
10        System.out.println("2. Tampilkan Semua Mahasiswa");
11        System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
12        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
13        System.out.println("5. Keluar");
14    }
15    // 2. Method untuk tambah mahasiswa
16    public static void tambahMahasiswa_2511531020 (ArrayList<Mahasiswa_2511531020> list, Scanner sc) {
17        System.out.print ("Masukan NIM: ");
18        String nim = sc.nextLine();
19        System.out.print ("Masukan Nama: ");
20        String nama = sc.nextLine();
21        System.out.print ("Masukan Prodi: ");
22        String prodi = sc.nextLine();
23        list.add (new Mahasiswa_2511531020 (nim, nama, prodi));
24        System.out.println("Mahasiswa berhasil ditambahkan.");
25    }
26    // 3. Method untuk tampilkan semua data
27    public static void tampilSemuaMahasiswa_2511531020 (ArrayList<Mahasiswa_2511531020> list) {
28        if (list.isEmpty()) {
29            System.out.println ("Daftar mahasiswa kosong.");
30            for (Mahasiswa_2511531020 mhs : list) {
31                System.out.println (mhs);
32            }
33        }
34    }
35
36    // 4. Method untuk hapus mahasiswa_2511531020 berdasarkan NIM
37    public static void hapusMahasiswa_2511531020 (ArrayList<Mahasiswa_2511531020> list, Scanner sc) {
38        System.out.print ("Masukkan NIM yang akan dihapus: ");
39        String nimHapus = sc.nextLine();
40        boolean removed = list.removeIf(mhs -> mhs.nim.equals (nimHapus));
41
42        if (removed) {
43            System.out.println("Data dengan NIM " + nimHapus + " berhasil dihapus.");
44        } else {
45            System.out.println("NIM tidak ditemukan.");
46        }
47    }
48    // 5. Method untuk cari mahasiswa berdasarkan NIM
49    public static void cariMahasiswa_2511531020 (ArrayList<Mahasiswa_2511531020> list, Scanner sc) {
50        System.out.print ("Masukkan NIM yang dicari: ");
51        String nimCari = sc.nextLine();
52        boolean ditemukan = false;
53
54        for (Mahasiswa_2511531020 mhs : list) {
55            if (mhs.nim.equals(nimCari)) {
56                System.out.println("Hasil Pencarian: " + mhs);
57                ditemukan = true;
58                break;
59            }
60        }
61
62        if (!ditemukan) {
63            System.out.println("NIM tidak ada.");
64        }
65    }
66
67    public static void main(String[] args) {
68        ArrayList<Mahasiswa_2511531020> mahasiswaList = new ArrayList<>();
69        Scanner scanner = new Scanner(System.in);
70        int choice;

```

```

70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103

```

```

    // ...
    do {
        tampilkanMenu_2511531020();
        System.out.print("Pilih menu: ");
        choice = scanner.nextInt();
        scanner.nextLine(); // Consume newline
        switch (choice) {
            case 1:
                tambahMahasiswa_2511531020(mahasiswaList, scanner);
                break;
            case 2:
                tampilSemuaMahasiswa_2511531020(mahasiswaList);
                break;
            case 3:
                hapusMahasiswa_2511531020(mahasiswaList, scanner);
                break;
            case 4:
                cariMahasiswa_2511531020(mahasiswaList, scanner);
                break;
            case 5:
                System.out.println("Keluar dari program.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (choice != 5);
    scanner.close();
}

```

**Gambar 2.10** Class 3 “MahasiswaDriver\_2511531020”

#### 4. Analisis Langkah

##### a. Tujuan Program

- Membuat aplikasi sederhana berbasis konsol untuk mengelola data mahasiswa.
- Menyediakan menu interaktif untuk menambah, menampilkan, menghapus, dan mencari mahasiswa berdasarkan NIM.
- Melatih penggunaan ArrayList, Scanner, serta konsep OOP dalam Java.

##### b. Struktur Program

- Package: pekan2\_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: MahasiswaDriver\_2511531020 → berfungsi sebagai driver class untuk menguji kelas Mahasiswa\_2511531020.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.
- Method tambahan:
  - tampilkanMenu\_2511531020() → menampilkan pilihan menu.

- tambahMahasiswa\_2511531020() → menambahkan data mahasiswa baru.
- tampilSemuamahasiswa\_2511531020() → menampilkan seluruh data mahasiswa.
- hapusmahasiswa\_2511531020() → menghapus mahasiswa berdasarkan NIM.
- cariMahasiswa\_2511531020() → mencari mahasiswa berdasarkan NIM.

#### c. Proses Eksekusi

- Program membuat ArrayList<Mahasiswa\_2511531020> untuk menyimpan daftar mahasiswa.
- Menggunakan Scanner untuk membaca input dari pengguna.
- Menampilkan menu berulang kali dengan do-while sampai pengguna memilih keluar (pilihan 5).
- Setiap pilihan menu dijalankan melalui switch-case:
  - Case 1: Tambah mahasiswa → meminta input NIM, nama, dan prodi, lalu menambahkan objek ke list.
  - Case 2: Tampilkan semua mahasiswa → mencetak isi list, atau pesan jika list kosong.
  - Case 3: Hapus mahasiswa → meminta NIM, lalu menghapus data dengan removeIf.
  - Case 4: Cari mahasiswa → meminta NIM, lalu mencari dengan loop dan menampilkan hasil.
  - Case 5: Keluar dari program.

#### d. Menampilkan Nilai Objek

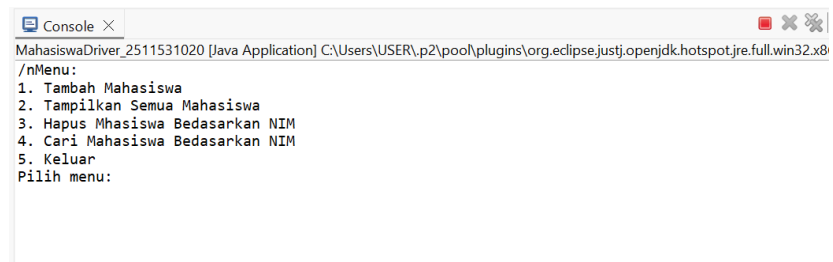
- Pemanggilan System.out.println(mhs) akan otomatis memanggil method toString() dari kelas Mahasiswa\_2511531020.
- Output berupa format: NIM: ..., Nama: ..., Prodi: ....



#### e . Analisis Konsep OOP dan Struktur Data

- Encapsulation: Data mahasiswa disimpan dalam objek Mahasiswa\_2511531020.
- Constructor: digunakan untuk menginisialisasi atribut mahasiswa saat objek dibuat.
- Method Overriding: toString() dioverride untuk menampilkan representasi objek yang lebih informatif.
- ArrayList: digunakan sebagai struktur data dinamis untuk menambah, menghapus, dan mencari mahasiswa.
- Control Flow: penggunaan do-while dan switch-case untuk mengatur alur menu interaktif.

#### 5. Output

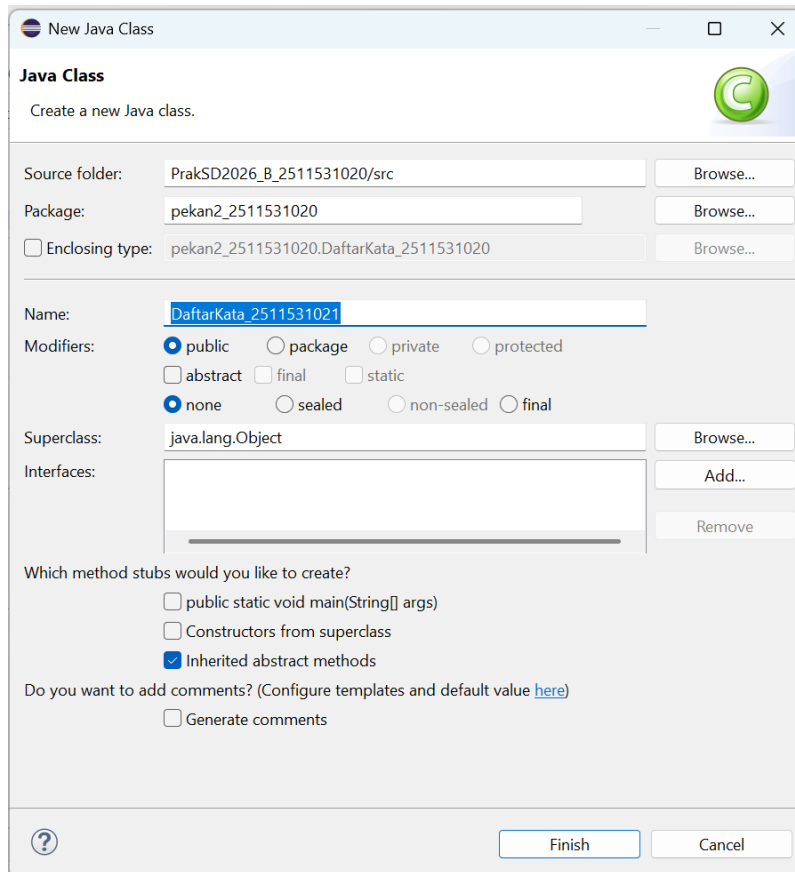


```
Console X
MahasiswaDriver_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe
/nMenu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu:
```

**Gambar 2.11** Output

#### 2.2.6 Class 4 “DaftarKata\_2511531020”

1. Membuat package pekan2\_2511531020 dan class DaftarKata\_2511531020



**Gambar 2.12** Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan2_2511531020;
2
3 import java.util.ArrayList;
4
5 public class DaftarKata_2511531020 {
6     private final ArrayList<String> data;
7
8     // Konstruktor: inisialisasi list kosong
9     public DaftarKata_2511531020() {
10         this.data = new ArrayList<>();
11     }
12
13     /**Menambahkan elemen di akhir list. */
14     public void tambah(String elemen) {
15         data.add(elemen);
16     }
17
18     /** Menambahkan elemen pada indeks tertentu (menyisipkan). */
19     public void tambahPada(int index, String elemen) {
20         data.add(index, elemen);
21     }
22
23     /**
24      * Mengubah elemen pada posisi 'index' menjadi 'nilaiBaru'.
25      * Bertindak sebagai "setter" untuk elemen tertentu. */
26     public void ubahElemen(int index, String nilaiBaru) {
27         data.set(index, nilaiBaru);
28     }
29
30     /** Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus. */
31     public String hapusElemen(int index) {
32         return data.remove(index);
33     }
34
35     /**
36      * Melakukan iterasi dan mencetak setiap elemen dalam format: [index] nilai
37      * (Metode ini tidak mengembalikan nilai; hanya demonstrasi iterasi).
38      */
39     public void iterasiCetak() {
40         for (int i = 0; i < data.size(); i++) {
41             System.out.print(data.get(i) + " ");
42         }
43     }
44
45     /** Mengambil elemen berdasarkan indeks. */
46     public String get(int index) {
47         return data.get(index);
48     }
49
50     // Representasi string agar mudah dicetak
51     @Override
52     public String toString() {
53         return data.toString();
54     }
55 }

```

**Gambar 2.13** Class 4 “DaftarKata\_2511531020”

#### 4. Analisis Langkah

##### a. Tujuan Program

- Membuat kelas DaftarKata sebagai wadah untuk menyimpan kumpulan string menggunakan ArrayList.
- Menyediakan operasi dasar seperti menambah, menyisipkan, mengubah, menghapus, dan mengambil elemen.
- Menunjukkan cara melakukan iterasi dan menampilkan isi list.

##### b. Struktur Program

- Package: pekan2\_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: DaftarKata\_2511531020 → berfungsi sebagai kelas utama untuk mengelola daftar kata.
- Atribut: `private final ArrayList<String> data` → list internal untuk menyimpan elemen string.
- Constructor: `DaftarKata_2511531020()` → menginisialisasi list kosong.
- Method utama:
  - `tambah(String elemen)` → menambahkan elemen di akhir list.
  - `tambahPada(int index, String elemen)` → menyisipkan elemen pada posisi tertentu.
  - `ubahElemen(int index, String nilaiBaru)` → mengganti elemen pada indeks tertentu.
  - `hapusElemen(int index)` → menghapus elemen dan mengembalikan nilai yang dihapus.
  - `iterasiCetak()` → mencetak isi list satu per satu.
  - `get(int index)` → mengambil elemen berdasarkan indeks.
  - `toString()` → menampilkan isi list dalam format string.

#### c. Proses Eksekusi

- Saat objek `DaftarKata_2511531020` dibuat, list internal data kosong.
- Pemanggilan `tambah("kata")` akan menambahkan string ke akhir list.
- Pemanggilan `tambahPada(1, "kataBaru")` akan menyisipkan string pada indeks ke-1.

- Pemanggilan `ubahElemen(0, "nilaiBaru")` akan mengganti elemen pertama dengan nilai baru.
- Pemanggilan `hapusElemen(2)` akan menghapus elemen pada indeks ke-2 dan mengembalikan nilai yang dihapus.
- Pemanggilan `iterasiCetak()` akan mencetak isi list satu per satu dengan loop.
- Pemanggilan `System.out.println(objekDaftarKata)` akan memanggil `toString()` dan menampilkan isi list dalam format `[elemen1, elemen2, ...]`.

d. Menampilkan Nilai Objek

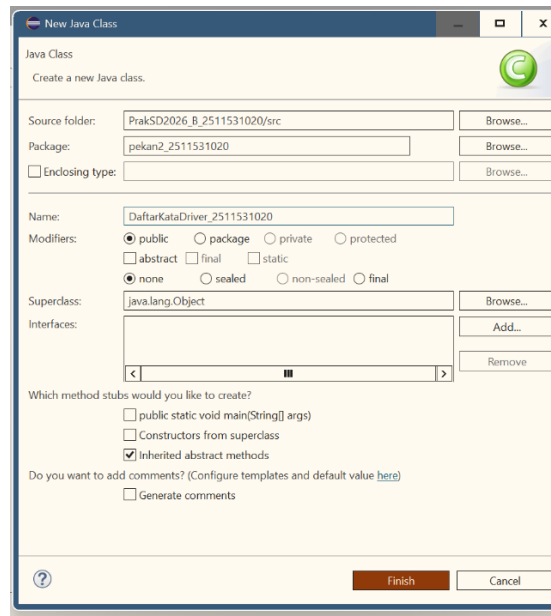
- Output dari `toString()` misalnya: `[satu, dua, tiga]`.
- Output dari `iterasiCetak()` misalnya: `satu dua tiga`.

e. Analisis Konsep OOP dan Struktur Data

- Encapsulation: list data disimpan sebagai atribut privat, hanya bisa diakses melalui method publik.
- Constructor: digunakan untuk menginisialisasi list kosong saat objek dibuat.
- Method Utility: menyediakan operasi dasar manipulasi data (`add`, `set`, `remove`, `get`).
- Dynamic Data Structure: `ArrayList` memungkinkan penambahan, penghapusan, dan pengubahan elemen secara fleksibel.
- Polymorphism: method `toString()` dioverride agar objek dapat ditampilkan dengan representasi yang lebih informatif.

### 2.2.7 Class 5 “DaftarKataDriver\_2511531020”

1. Membuat package pekan2\_2511531020 dan class DaftarKataDriver\_2511531020



**Gambar 2.14** Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```
1 package pekan2_2511531020;
2
3 public class DaftarKataDriver_2511531020 {
4
5     public static void main(String[] args) {
6         DaftarKata_2511531020 al = new DaftarKata_2511531020();
7
8         // Menambah elemen (akhir)
9         al.tambah("Kami");
10        al.tambah("Informatika");
11
12        // Menyisipkan elemen pada indeks 1
13        al.tambahPada(1, "Mahasiswa");
14
15        // Cetak isi awal
16        System.out.println("Awal          : " + al);
17
18        // Mengubah elemen (index 1)
19        al.ubahElemen(1, "Departemen");
20        System.out.println("Setelah Ubah: " + al);
21
22        // Menghapus elemen (hapus index 0)
23        String terhapus = al.hapusElemen(0);
24        System.out.println("Terhapus   : " + terhapus);
25        System.out.println("Setelah Hapus: " + al);
26
27        // Iterasi pada ArrayList (cetak setiap elemen)
28        System.out.print("Iterasi: ");
29        al.iterasiCetak();
30        System.out.println();
31    }
32 }
33
```

**Gambar 2.15** “DaftarKataDriver\_2511531020”

4. Analisis Langkah

#### a. Tujuan Program

- Menguji kelas `DaftarKata_2511531020` dengan membuat objek daftar kata.
- Melakukan operasi penambahan, penyisipan, pengubahan, penghapusan, serta iterasi elemen dalam list.
- Menampilkan hasil setiap operasi ke konsol untuk memperlihatkan perubahan isi list.

#### b. Struktur Program

- Package: `pekan2_2511531020` → menunjukkan program berada dalam paket tertentu.
- Class: `DaftarKataDriver_2511531020` → berfungsi sebagai driver class untuk menguji kelas `DaftarKata_2511531020`.
- Method utama: `public static void main(String[] args)` → titik masuk eksekusi program.

#### c. Proses Eksekusi

- Membuat objek `DaftarKata_2511531020` `al = new DaftarKata_2511531020();` → list kosong siap digunakan.
- Menambahkan elemen dengan `tambah("Kami")` dan `tambah("Informatika")` → isi list menjadi `[Kami, Informatika]`.
- Menyisipkan elemen pada indeks ke-1 dengan `tambahPada(1, "Mahasiswa")` → isi list menjadi `[Kami, Mahasiswa, Informatika]`.
- Mencetak isi awal dengan `System.out.println("Awal : " + al);` → output: `Awal : [Kami, Mahasiswa, Informatika]`.
- Mengubah elemen indeks ke-1 dengan `ubahElemen(1, "Departemen")` → isi list menjadi `[Kami, Departemen, Informatika]`.

- Menghapus elemen indeks ke-0 dengan `hapusElemen(0)` → elemen Kami terhapus, list menjadi [Departemen, Informatika].
- Menampilkan elemen yang dihapus dan isi list setelah penghapusan.
- Melakukan iterasi dengan `iterasiCetak()` → mencetak elemen satu per satu: Departemen Informatika.

d. Menampilkan Nilai Objek

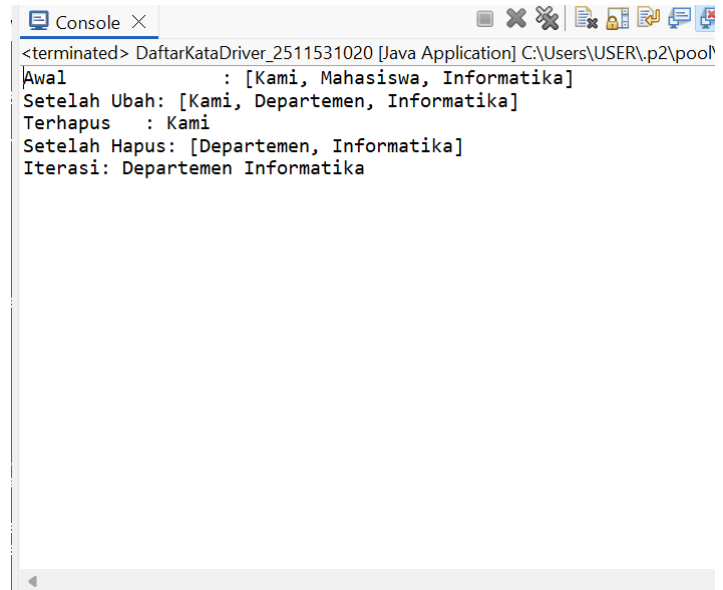
- Output dari `toString()`: [Kami, Mahasiswa, Informatika], [Kami, Departemen, Informatika], [Departemen, Informatika].
- Output dari iterasi: Departemen Informatika.

e. Analisis Konsep OOP dan Struktur Data

- Encapsulation: list data dikelola melalui method publik, atribut internal tidak diakses langsung.
- Constructor: menginisialisasi list kosong saat objek dibuat.
- Method Utility: `tambah`, `tambahPada`, `ubahElemen`, `hapusElemen`, `get`, dan `iterasiCetak` menunjukkan operasi dasar manipulasi data.
- Dynamic Data Structure: penggunaan `ArrayList` memungkinkan penambahan, penghapusan, dan pengubahan elemen secara fleksibel.
- Polymorphism: pemanggilan `System.out.println(al)` memanfaatkan override `toString()` untuk menampilkan isi objek dengan format yang lebih informatif.



## 5. Output

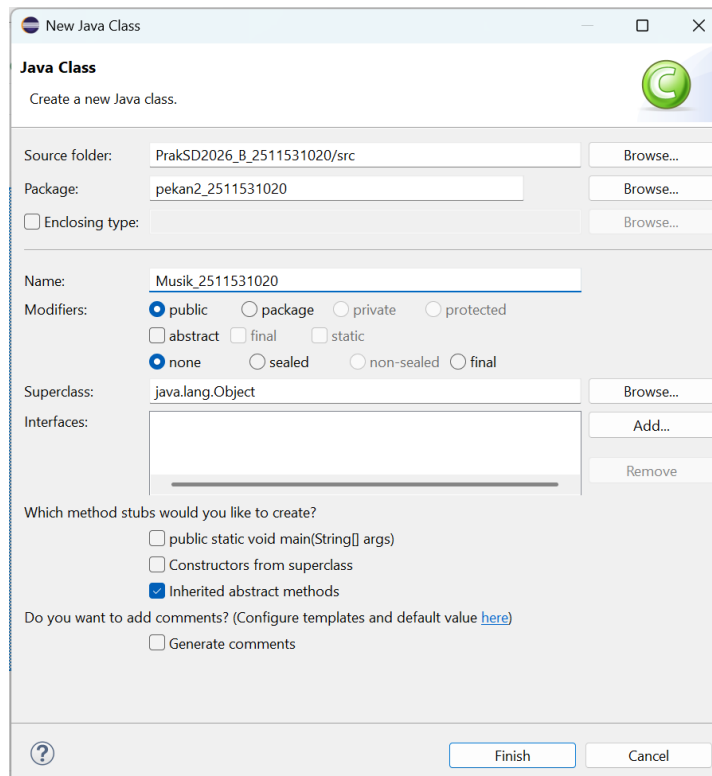


```
<terminated> DaftarKataDriver_2511531020 [Java Application] C:\Users\USER\p2\pool\
Awal : [Kami, Mahasiswa, Informatika]
Setelah Ubah: [Kami, Departemen, Informatika]
Terhapus : Kami
Setelah Hapus: [Departemen, Informatika]
Iterasi: Departemen Informatika
```

**Gambar 2.16** Output

## 2.3 Tugas Praktikum

### 2.3.1 Class Musik\_2511531020



**Gambar 2.17** Membuat Nama package dan class

#### 1. Kode Program

```
1 package pekan2_2511531020;
2
3 public class Musik_2511531020 {
4     private String judul_1020;
5     private String penyanyi_1020;
6     private int durasi_1020;
7
8     // Constructor
9     public Musik_2511531020(String judul_1020, String penyanyi_1020, int durasi_1020) {
10         this.judul_1020 = judul_1020;
11         this.penyanyi_1020 = penyanyi_1020;
12         this.durasi_1020 = durasi_1020;
13     }
14
15     // Getter
16     public String getJudul_1020() {
17         return judul_1020;
18     }
19
20     public String getPenyanyi_1020() {
21         return penyanyi_1020;
22     }
23
24     public int getDurasi_1020() {
25         return durasi_1020;
26     }
27
28     // Setter
29     public void setJudul_1020(String judul_1020) {
30         this.judul_1020 = judul_1020;
31     }
32
33     public void setPenyanyi_1020(String penyanyi_1020) {
34         this.penyanyi_1020 = penyanyi_1020;
35     }
36 }
```

**Gambar 2.18** class Musik\_2511531020

## 2. Analisis Langkah

### a. Tujuan

Kelas Musik\_2511531020 dibuat sebagai ADT (Abstract Data Type) yang berfungsi menjadi cetak biru objek lagu. Tujuannya adalah untuk mendefinisikan atribut-atribut penting dari sebuah lagu (judul, penyanyi, durasi) dan menyediakan method untuk mengakses serta memodifikasi data tersebut.

### b. Struktur

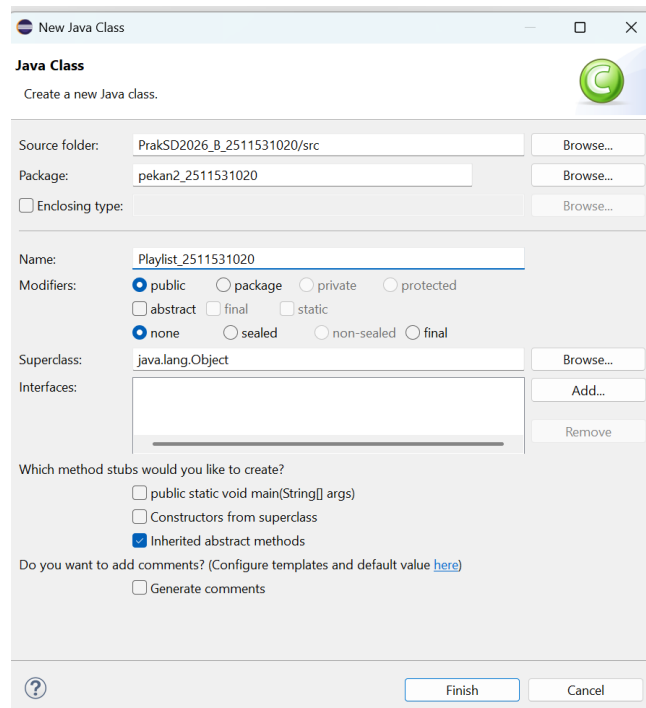
- Atribut privat:
  - judul\_1020 → menyimpan judul lagu.
  - penyanyi\_1020 → menyimpan nama penyanyi.
  - durasi\_1020 → menyimpan durasi lagu dalam detik.
- Constructor: digunakan untuk menginisialisasi objek baru dengan data lagu yang dimasukkan.
- Getter (Selektor): method untuk mengambil nilai dari setiap atribut.
- Setter (Mutator): method untuk mengubah nilai atribut jika diperlukan.
- Method toString(): dioverride agar objek lagu bisa ditampilkan dalam format string yang informatif.

### c. Proses Eksekusi

- Saat pengguna menambahkan lagu melalui kelas driver, objek baru dari Musik\_2511531020 dibuat dengan memanggil constructor.
- Data lagu (judul, penyanyi, durasi) disimpan dalam atribut privat.
- Jika program ingin menampilkan informasi lagu, method toString() dipanggil sehingga objek ditampilkan dalam bentuk teks.

- Getter dan setter memungkinkan pengambilan atau perubahan data lagu tanpa langsung mengakses atribut privat.
- d. Peran dalam Program
- Kelas ini tidak menghasilkan output langsung, melainkan menyediakan struktur data yang akan digunakan oleh kelas driver.
  - Semua operasi input/output (tambah, lihat, hapus, cek kapasitas) dilakukan di kelas Playlist\_2511531020.
  - Kelas Musik\_2511531020 berperan sebagai model data yang memastikan setiap lagu memiliki format dan atribut yang konsisten.
- e. Konsep OOP yang Diterapkan
- Encapsulation: atribut disimpan privat, hanya bisa diakses lewat getter dan setter.
  - Constructor: memudahkan pembuatan objek dengan data lengkap.
  - Polymorphism (method overriding): toString() dioverride agar objek dapat ditampilkan dengan cara yang lebih informatif.
  - Abstraction: pengguna tidak perlu tahu detail penyimpanan data, cukup menggunakan method yang tersedia.

### 2.3.2 Class Playlist\_2511531020



**Gambar 2.19** Membuat Nama package dan class

#### 1. Kode Program

```
1 package pekan2_2511531020;
2
3 import java.util.ArrayList;
4 import java.util.Scanner;
5
6 public class Playlist_2511531020 {
7     private ArrayList<Musik_2511531020> daftarLagu_1020;
8
9     public Playlist_2511531020() {
10         daftarLagu_1020 = new ArrayList<>();
11     }
12
13     // Tambah Lagu
14     public void tambahLagu(Musik_2511531020 lagu) {
15         daftarLagu_1020.add(lagu);
16         System.out.println("Data berhasil ditambahkan!");
17     }
18
19     // Lihat Playlist
20     public void lihatPlaylist() {
21         if (daftarLagu_1020.isEmpty()) {
22             System.out.println("Playlist kosong.");
23         } else {
24             System.out.println("=== Daftar Lagu ===");
25             for (int i = 0; i < daftarLagu_1020.size(); i++) {
26                 System.out.println((i + 1) + ". " + daftarLagu_1020.get(i));
27             }
28         }
29     }
30
31     // Hapus Lagu berdasarkan indeks
32     public void hapusLagu(int index) {
33         if (index >= 0 && index < daftarLagu_1020.size()) {
34             Musik_2511531020 terhapus = daftarLagu_1020.remove(index);
35             System.out.println("Lagu \"" + terhapus.getJudul_1020() + "\" berhasil dihapus.");
```

```

36     } else {
37         System.out.println("Indeks tidak valid.");
38     }
39 }
40
41 // Cek Kapasitas
42 public void cekKapasitas() {
43     System.out.println("Jumlah total lagu: " + daftarLagu_1020.size());
44 }
45
46 // Main Program
47 public static void main(String[] args) {
48     Playlist_2511531020 playlist = new Playlist_2511531020();
49     Scanner sc = new Scanner(System.in);
50     int pilihan;
51
52     do {
53         System.out.println("\n== Playlist Musik NIM: 2511531020 ==");
54         System.out.println("1. Tambah Lagu");
55         System.out.println("2. Lihat Playlist");
56         System.out.println("3. Hapus Lagu");
57         System.out.println("4. Cek Kapasitas");
58         System.out.println("5. Keluar");
59         System.out.print("Pilihan: ");
60         pilihan = sc.nextInt();
61         sc.nextLine(); // konsumsi newline
62
63         switch (pilihan) {
64             case 1:
65                 System.out.print("Masukkan Judul: ");
66                 String judul = sc.nextLine();
67                 System.out.print("Masukkan Penyanyi: ");
68                 String penyanyi = sc.nextLine();
69                 System.out.print("Masukkan Durasi (detik): ");
70                 int durasi = sc.nextInt();
71
72                 // durasi = sc.nextInt();
73                 sc.nextLine();
74                 playlist.tambahLagu(new Musik_2511531020(judul, penyanyi, durasi));
75                 break;
76             case 2:
77                 playlist.lihatPlaylist();
78                 break;
79             case 3:
80                 playlist.lihatPlaylist();
81                 System.out.print("Masukkan nomor lagu yang akan dihapus: ");
82                 int index = sc.nextInt() - 1;
83                 sc.nextLine();
84                 playlist.hapusLagu(index);
85                 break;
86             case 4:
87                 playlist.cekKapasitas();
88                 break;
89             case 5:
90                 System.out.println("Keluar dari program.");
91                 break;
92             default:
93                 System.out.println("Pilihan tidak valid.");
94         }
95     } while (pilihan != 5);
96
97     sc.close();
98 }

```

**Gambar 2.20** class DriverMobil

## 2. Analisis Langkah

### a. Tujuan

Kelas Playlist\_2511531020 berfungsi sebagai driver class yang mengatur jalannya program. Tujuannya adalah:

- Mengelola daftar lagu menggunakan ArrayList<Musik\_2511531020>.

- Menyediakan menu interaktif agar pengguna bisa menambah, melihat, menghapus, dan mengecek jumlah lagu.
- Menampilkan hasil operasi ke konsol sehingga pengguna dapat berinteraksi langsung dengan playlist.

#### b. Struktur

- Atribut:
  - `ArrayList<Musik_2511531020>`  
`daftarLagu_1020` → menyimpan kumpulan objek lagu.
- Method utama:
  - `tambahLagu()` → menambahkan objek lagu ke playlist.
  - `lihatPlaylist()` → menampilkan semua lagu dengan looping.
  - `hapusLagu()` → menghapus lagu berdasarkan indeks.
  - `cekKapasitas()` → menampilkan jumlah lagu dengan `.size()`.
- Method `main()`: berisi menu interaktif dengan pilihan 1–5 (Tambah, Lihat, Hapus, Cek Kapasitas, Keluar).

#### c. Proses Eksekusi

- Program dimulai dari `main()`.
- Objek playlist dibuat: `Playlist_2511531020 playlist = new Playlist_2511531020();`.
- Menu ditampilkan berulang menggunakan `do-while`.

- Tambah Lagu: pengguna memasukkan judul, penyanyi, durasi → dibuat objek Musik\_2511531020 → ditambahkan ke ArrayList.
- Lihat Playlist: program melakukan looping pada ArrayList dan menampilkan semua lagu dengan format dari toString().
- Hapus Lagu: program menampilkan daftar lagu, meminta nomor indeks, lalu menghapus lagu sesuai pilihan.
- Cek Kapasitas: program menampilkan jumlah lagu dengan daftarLagu\_1020.size().
- Keluar: program berhenti setelah pengguna memilih opsi 5.

d. Contoh Output

- Tambah Lagu:

Code

Masukkan Judul: Cincin

Masukkan Penyanyi: Hindia

Masukkan Durasi (detik): 300

Data berhasil ditambahkan!

- Lihat Playlist:

Code

=== Daftar Lagu ===

1. Judul: Cincin, Penyanyi: Hindia, Durasi: 300 detik

- Hapus Lagu:

Code

Lagu "Cincin" berhasil dihapus.



- Cek Kapasitas:

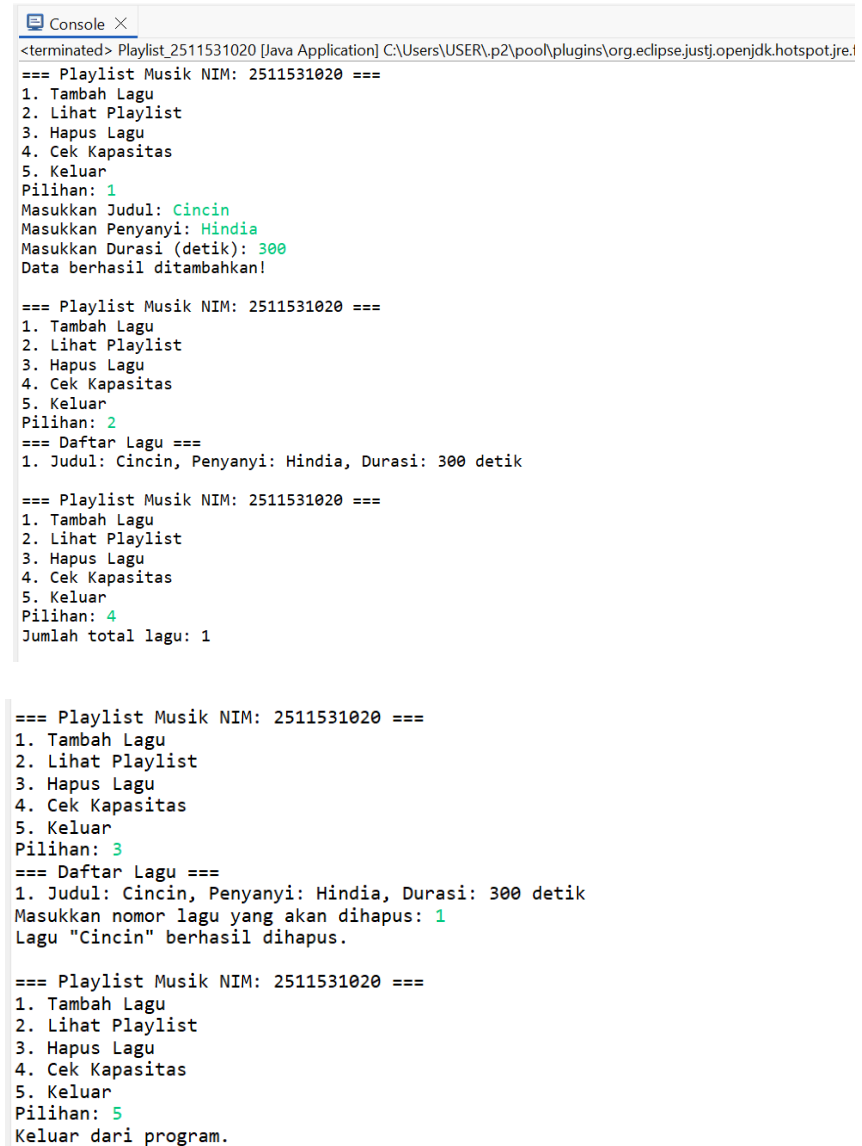
Code

Jumlah total lagu: 1

e. Analisis Konsep OOP dan Struktur Data

- Encapsulation: daftar lagu dikelola melalui method publik, tidak diakses langsung.
- Constructor: menginisialisasi playlist kosong saat objek dibuat.
- Polymorphism: pemanggilan `System.out.println(lagu)` memanfaatkan override `toString()` dari kelas Musik.
- Abstraction: pengguna cukup memilih menu tanpa perlu tahu detail implementasi `ArrayList`.
- Dynamic Data Structure: penggunaan `ArrayList` memungkinkan playlist bertambah atau berkurang sesuai kebutuhan.

### 3. Output



```
Console X
<terminated> Playlist_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.i

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 1
Masukkan Judul: Cincin
Masukkan Penyanyi: Hindia
Masukkan Durasi (detik): 300
Data berhasil ditambahkan!

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 2
=== Daftar Lagu ===
1. Judul: Cincin, Penyanyi: Hindia, Durasi: 300 detik

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 4
Jumlah total lagu: 1

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 3
=== Daftar Lagu ===
1. Judul: Cincin, Penyanyi: Hindia, Durasi: 300 detik
Masukkan nomor lagu yang akan dihapus: 1
Lagu "Cincin" berhasil dihapus.

=== Playlist Musik NIM: 2511531020 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 5
Keluar dari program.
```

**Gambar 2.21** Output

## BAB III

### KESIMPULAN

Kesimpulan dari seluruh praktikum dan tugas yang telah dikerjakan adalah bahwa penggunaan ArrayList dalam Java memberikan solusi efektif untuk mengelola data secara dinamis, berbeda dengan array konvensional yang bersifat statis. Melalui implementasi kelas Musik\_2511531020 sebagai ADT dan kelas Playlist\_2511531020 sebagai driver, mahasiswa dapat memahami bagaimana konsep Object-Oriented Programming (OOP) diterapkan dalam membangun program yang modular dan fleksibel. Kelas Musik berperan sebagai cetak biru data lagu dengan atribut, constructor, getter, setter, dan method toString(), sementara kelas Playlist mengatur logika program, menyediakan menu interaktif, dan menampilkan output ke konsol.

Program yang dihasilkan mampu menambah, menampilkan, menghapus, serta menghitung jumlah lagu dalam playlist sesuai instruksi tugas. Praktikum ini juga menegaskan pentingnya prinsip OOP seperti *encapsulation*, *abstraction*, dan *polymorphism* dalam pengembangan perangkat lunak. Dengan struktur yang rapi, aturan penamaan sesuai NIM, serta fungsionalitas yang berjalan baik, tugas ini memenuhi seluruh kriteria penilaian dan memberikan pengalaman nyata dalam mengelola data menggunakan Java Collections Framework.

Secara keseluruhan, praktikum ini berhasil melatih keterampilan teknis sekaligus memperkuat pemahaman konsep dasar struktur data dan pemrograman berorientasi objek. Hal ini memberikan bekal penting bagi mahasiswa dalam mengembangkan program yang lebih kompleks dan terstruktur di masa mendatang.

## DAFTAR PUSTAKA

- [1] Oracle, "Class ArrayList (Java Platform SE 8)," Oracle Documentation, 2015. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- [2] Oracle, "The Java™ Tutorials: Collections," Oracle, 2015. [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/index.html>
- [3] Cay S. Horstmann, *Big Java: Early Objects*, 6th ed., Wiley, 2016.
- [4] Herbert Schildt, *Java: The Complete Reference*, 11th ed., McGraw-Hill Education, 2018.
- [5] Paul Deitel and Harvey Deitel, *Java How to Program*, 11th ed., Pearson, 2017.